Names: Norman Nomie, Drew Inglesby, Jacob Munly, Jack Dunlap
Class: CS362
Assignment: Project Proposal

– Textplorer –

Project Milestone #1 (Revised 2/26/26)

Team Members and Roles
Jacob Munly - Project Manager: In charge of creating the plan/schedule for the project, organizing the repository, assigning tasks, and ensuring deadlines are met.

Jack Dunlap - QA/Documentation: Responsible for maintaining the project's documents, including repository documents and submission assignments and ensuring all criteria are met.

Drew Inglesby - UX/UI designer: Responsible for the conceptual layout of the game design, designing how users interact with the game, and the online aspect of the project.

Norman Nomie - Lead Developer: Responsible for overseeing the software implementation, planning out the programming logic, and assigning pieces of code to team members.

Project Artifacts:
GitHub Repository Link: https://github.com/gilbertboys/Textplorer

Communication Plan:
We will be using both Discord and text messaging to communicate with each other while we work on this project. Discord is useful for being able to share files and images, and text messaging is great for communicating when we are away from our computers. We will communicate with each other whenever we feel stuck/confused, or one of us is making a major design choice/overhaul to our code. Beyond meeting with each other during each lecture, we will regularly schedule in-person meetings and remote meetings via Discord to discuss our project and work together. Discord works great for meeting remotely, as it supports screen sharing.

Product Description

Abstract: Textplorer is an innovative 2D platforming engine designed to simplify game design by bridging the gap between simple text editing and interactive level generation. Traditional level editors often rely on complex, nested menus that create a steep learning curve for casual creators. Textplorer bypasses this friction by using a coordinate-based parsing system to transform standard .txt files into playable environments in real-time. By leveraging the universal skill of typing, Textplorer offers a lightweight, accessible, and infinitely replayable experience where the user's vocabulary becomes the blueprint for their gameplay.

Goal: Our goal is to create a simple yet fun platforming game where users can easily create, share, and play levels. We want our game to be accessible to anyone with a computer and an internet connection. Games today often have massive file sizes and require powerful hardware, and which leads to many people not being able to experience them. We want to create a lightweight and intuitive game that allows users to create and play 2D platforming levels, with only their imagination as a limit.

Current Practice: Modern platforming games (like Super Mario Maker 2 and Fortnite Creative) that include level editors often run into the issue of having giant nested menus where users can select from every object in the game. While this approach can work for die-hard fans of a game, it is often unintuitive for new users.

Novelty: What makes our approach to user generated platforming levels special, is that all users already have experience in Textplorer level creation, as anyone with a computer has typed out words. Everybody that owns a laptop or a computer knows how to type, which means that they can all create levels to play and share.

Effect: This project will impact users who enjoy simple video games or those who love the unique platformer experience. Our hope is for Textplorer to be a social game, where friends share fun levels that they have created or previously played. Textplorer offers a new way to create and play platforming challenges. The game allows users to turn text into a playable map and gives the game a near-infinite replayability, with the only limit being user creativity. Our hope for the overall effect of our project is to spread joy to our users and allow them to be creative.

Technical Approach: The development of Textplorer focuses on creating a seamless pipeline between static data (text) and dynamic gameplay (physics). Our approach is divided into three core phases:

1. Data Parsing & Coordinate Mapping

The engine treats each .txt file as a two-dimensional coordinate grid. Using javascript file-stream libraries, the program parses the file character by character. Each character maps to a specific game object based on a predefined legend (e.g., | for a solid wall, _ for player start, ^ for hazards). This allows for precise level reconstruction where the text file's layout directly mirrors the in-game environment.

2. Physics & Collision Logic
Since the project moves beyond a Command Line Interface (CLI), we are implementing a custom physics engine to handle:
-AABB (Axis-Aligned Bounding Box) Collision: To detect interactions between the player and the environment blocks.
-Kinematics: Implementing gravity constants and velocity vectors to ensure movement feels fluid rather than rigid.
-State Management: Tracking whether a player is grounded, jumping, or interacting with a "goal" tile to trigger level completion.

3. Systems Architecture & UI
The application is built using a state-based architecture to manage transitions between the Main Menu, Level Loader, and Active Gameplay.
Level Manager: A module that scans the local directory for available .txt files and populates the UI list.
File I/O: An integrated uploader that validates user-created text files to ensure they contain necessary elements (like a start and end point) before allowing them to be played.

4. Development Environment
The project is authored in Visual Studio Code. We utilize Git/GitHub for version control and a modular programming approach, allowing the Software Lead to assign specific components (like UI or Physics) to different team members simultaneously.

Risks: The most significant risk with developing our project is that our group doesn't have much experience developing games with graphics and physics. Most of our experience as developers so far has been with command line interfaces, which don't incorporate graphics or physics. We will minimize this risk by taking advantage of the plethora of beginner game development guides on YouTube.

Features
Major Features (MVP-Minimum Viable Product)

1. Physics & Movement: This feature is the core of the project, allowing users to control and interact with the levels within the game. The physics must be intuitive and consistent

for users, and create exciting gameplay. Physics & movement includes jumping, gravity, moving left/right as well as how certain character blocks interact with the player.
2. Level Generation & Display: The program must be able to transform txt files from a "levels" folder into playable, interactive, real-time platforming levels.
3. File Upload: This feature allows users to upload txt files to the program, allowing users to create, share, and play their own levels.
4. User Interface: The user interface for a minimum viable product would include a main menu screen with options to Play, Upload, or Exit. Beyond that, the user would be able to pause while playing a level to bring up a pause screen that freezes the physics of a level and lets them either select Continue or Exit To Main Menu.

Stretch Goals:

1. Time Attack Gamemode: An alternate gamemode that a player can select from the main menu that adds a time limit to levels. If the timer reaches 0 during play, the user would fail the level and be forced to restart.
2. Visual Themes: The game would include selectable visual themes that would change the look of the game from a simple black on white color scheme to a variety of choices. An example of one of these visual themes would be a "Sky" theme, where the background is sky blue and the foreground is white like a cloud.

Project Milestone #2 (Revised 2/10/26)

1. Use Cases (Functional Requirements)

As a team of four, here are four distinct use cases covering the core functionality of Textplorer.

Use Case 1: Level Creation and Loading (Jacob Munly)

- **Actors:** Level Creator (User)
- **Triggers:** User places a .txt file in the designated "levels" folder.
- **Preconditions:** The application is running; the .txt file follows the coordinate-based legend.
- **Postconditions:** The engine successfully parses the text and renders a playable 2D environment.
- **Success Steps:**
    1. User opens a standard text editor.
    2. User types a level layout using symbols like | for walls and ^ for hazards.
    3. User saves the file as level1.txt in the levels directory.
    4. User selects "Play" from the Textplorer main menu.
    5. The engine parses the file character-by-character and generates the map.
- **Extensions:** User can share the .txt file with a friend via Discord.

- **Exceptions:** File is missing a "start" or "end" point; the File I/O module throws a validation error.

Use Case 2: Player Movement and Physics (Norman Nomie)

- **Actors:** Player
- **Triggers:** Player presses movement keys (e.g., Arrows or WASD).
- **Preconditions:** A level has been successfully loaded and the game state is "Active Gameplay".
- **Postconditions:** The player character moves fluidly according to gravity and collision rules.
- **Success Steps:**
    1. Player presses the "Jump" key.
    2. Physics engine applies a velocity vector to the character.
    3. The engine checks for AABB collisions with environment blocks.
    4. The character lands on a solid | block and the state updates to "grounded".
- **Extensions:** Player hits a hazard ^ and the level restarts.
- **Exceptions:** Player falls out of the map bounds; the game resets the player to the starting coordinate.

Use Case 3: Navigating the User Interface (Drew Inglesby)

- **Actors:** User
- **Triggers:** Launching the application or pressing the "Pause" key.
- **Preconditions:** Application is compiled and running.
- **Postconditions:** User successfully transitions between different game states.
- **Success Steps:**
    1. User views the Main Menu and clicks "Play".
    2. During gameplay, user presses 'Esc' to pause.
    3. The physics engine freezes all movement and velocity vectors.
    4. User selects "Exit to Main Menu".
- **Extensions:** User selects a "Visual Theme" (e.g., Sky Theme) from a settings sub-menu.
- **Exceptions:** Mouse click occurs outside of button bounds; no action is taken.

Use Case 4: Level Completion and Validation (Jack Dunlap)

- **Actors:** Player
- **Triggers:** Player character overlaps with the "Goal" tile.
- **Preconditions:** Player is in "Active Gameplay" mode.
- **Postconditions:** Game registers a win and returns the player to the level selection screen.
- **Success Steps:**
    1. Player navigates the platforming challenges.
    2. Player reaches the coordinate mapped to the "end point".
    3. The State Management module detects the "Goal" interaction.

4. A "Level Complete" message is displayed.
- **Extensions (Stretch):** The "Time Attack" timer stops, and the user's time is recorded.
- **Exceptions:** Player reaches the goal but the file validation was bypassed; the game fails to trigger the win state.

---

2. Non-functional Requirements

- **Usability (Accessibility):** The level creation process must require no specialized software other than a basic text editor, leveraging "the universal skill of typing" to ensure a low barrier to entry.
- **Performance (Lightweight Execution):** The game must maintain a consistent 60 FPS on standard laptop hardware by utilizing efficient Javascript file-streaming and custom AABB collision logic rather than resource-heavy commercial engines.
- **Portability (Web Readiness):** The game must be playable via a standard web browser.

---

3. External Requirements

- **Robustness:** The Textplorer engine will include a robust File I/O validation layer. If a user uploads a .txt file with unrecognizable characters or missing exit goals, the system will provide a clear error message in the UI rather than crashing.
- **Installability & Distribution:** For the web-based version, we will host the compiled project on **GitHub Pages**. For the standalone version, we will provide a ZIP archive containing the executable and a levels/ folder for easy "plug-and-play" access.
- **Buildability:** The repository will include a README.md and a Makefile (or CMake configuration). This ensures that any user with a standard web browser (Chrome) and VS Code can clone the repo and build the source code locally.
- **Resource Alignment:** The scope is tailored to a four-person team. By dividing the project into Physics, UI, Documentation, and Management, we ensure that the custom physics engine, our highest technical hurdle, is the primary focus.

---

4. Team Process Description

Software Toolset

- **JavaScript & VS Code:** Chosen because it is the team's primary area of expertise. VS Code’s modular extensions allow for efficient Javascript debugging.
- **Phaser:** An open-source 2D game framework specifically designed for making games that run in a web browser. Should make the jump to game development and 2D kinematics easier
- **Git/GitHub:** Used for version control and task tracking to allow simultaneous development of Physics and UI modules.
- **Discord:** Primary hub for file sharing, screen sharing during remote coding sessions, and rapid communication.

Roles and Justification

- **Jacob Munly (Project Manager):** Needed to keep the repository organized and ensure the four-person team meets weekly deadlines.
- **Norman Nomie (Software Lead):** Crucial for designing the "logic blueprint," especially since the team is transitioning from CLI to graphics.
- **Drew Inglesby (UX/UI Designer):** Responsible for making the "text-to-game" transition intuitive for users who might find traditional editors complex.
- **Jack Dunlap (Documentation):** Ensures the living document and technical requirements are updated, which is vital for a modular JavaScript project.

Weekly Schedule & Milestones

Week	Milestone	Responsibility
W1	Come up with project idea	Jacob
W2	Solidify project abstract and goal, research current practice	Norman
W3	Design/sketch what a complete Textplorer would look like.	Norman/Jacob
W4	Finalize software architecture	Drew
W5	Create “Hello World” website with Phaser Acade test physics and tech stack	Jack/Jacob
W6	Begin programming game logic, txt file -> tilemap paser.	Team
W7	Error handling: Validation for "broken" text files is implemented.	Jack
W8	Visual Themes: Sky theme and Dark mode toggles functional.	Drew

W9	Web Deployment: Project compiled to WebAssembly and hosted.	Norman
W10	Final Polish: Bug fixes and documentation finalization.	Team

Major Risks

1. **Physics Inexperience:** The team's background is in CLI, not 2D kinematics. We will mitigate this using tutorials and modular testing.
2. **Framework Learning Curve (The "New Environment" Risk):** Moving from raw C++ to a framework like Phaser (JavaScript) introduces a "syntax shock." The way these frameworks handle the "Game Loop" and "State Management" is different from the linear logic of standard C++ assignments.
3. **Scope Creep:** Stretch goals like "Time Attack" might distract from the core physics. We will not start these until the MVP is fully robust.

External Feedback

External feedback will be most useful from Week 6 onwards, once the MVP is functional. We will share the GitHub Pages link with classmates and friends to see if they find the "text-to-level" concept intuitive without instructions. Their feedback on "game feel" (gravity and jump height) will guide our physics tuning in the final weeks.

Project Milestone 3

1. SOFTWARE ARCHITECTURE

System Overview Textplorer follows a Client-Server Architecture. The frontend is built using the Phaser 3 framework, which handles the game loop and rendering. The backend (hosted on Heroku) serves the static assets and provides a simple Node.js entry point to handle level file uploads.

Major Components

- Phaser Game Engine: Manages the core lifecycle: Preload (assets), Create (parsing), and Update (input/physics).
- Level Parser Utility: A standalone JS module that reads strings/files and converts them into a 2D array of tile IDs.
- Physics Module (Arcade Physics): Phaser’s built-in engine used for AABB collisions and gravity.
- State Manager (Scenes): Transitions between Boot, MainMenu, and LevelScene.

- Heroku/Node Server: Serves the application and provides a basic API endpoint for level uploads.

Interfaces

- Upload API: The UI sends a .txt file via a POST request; the server validates and returns the raw text content.
- Parser to Scene: The Parser receives a string and returns a JSON-compatible grid to the LevelScene.
- Event Bus: Phaser's internal emitter handles communication between the UI (HUD) and the physics-driven game world.

Data Storage

- Client-Side: Level data is stored in memory as 2D arrays during gameplay.
- Server-Side: Initial bundled levels are stored as static .txt files in a /levels directory on the server.
- Browser LocalStorage: Used to save user-created levels or high scores without requiring a full database.

Assumptions

- Browser Compatibility: We assume users are on modern browsers supporting WebGL and the File API.
- Fixed Resolution: The game logic assumes a consistent aspect ratio for coordinate mapping.

Architecture Decisions and Alternatives

1. Hosting: Heroku vs. GitHub Pages
- Chosen: Heroku. Pros: Allows for a Node.js backend to handle future database integration. Cons: Higher setup complexity.
- Alternative: GitHub Pages. Pros: Free, zero-config for static sites. Cons: No backend capability.
2. Physics: Phaser Arcade vs. Matter.js
- Chosen: Arcade Physics. Pros: Perfect for simple AABB logic; high performance.
- Alternative: Matter.js. Pros: Sophisticated body shapes. Cons: Unnecessary overhead for a grid-based game.

---

2. SOFTWARE DESIGN

Component: Scenes (MainScene, UIScene)

- Units: Inherit from Phaser.Scene.

- Responsibilities: Manage the game loop and visual layers.

Component: Level Parser

- Units: Parser.js.
- Responsibilities: Uses regex and string splitting to map characters (e.g., #) to Phaser Tile IDs.

Component: Player Controller

- Units: Player.js (Sprite).
- Responsibilities: Extends Phaser.Physics.Arcade.Sprite. Handles keyboard input and velocity updates.

Component: File Handler

- Units: FileUploader.js.
- Responsibilities: Uses the Browser File API to read local user files and trigger the Parser.

Component: Server

- Units: server.js (Express).
- Responsibilities: Minimal Node.js script to serve the index.html and assets.

---

3. CODING GUIDELINE

- Guideline: Airbnb JavaScript Style Guide.
- Reasoning: It is the most widely adopted standard for modern JS, focusing on readability and preventing common bugs.
- Enforcement: We will hold each other accountable when reviewing each other's code to ensure we are within the style guidelines

---

4. PROCESS DESCRIPTION

i. Risk Assessment

Risk: Heroku Deployment

- Likelihood: High | Impact: Medium
- Evidence: Team has never used Heroku, we were recommended it by the TA so hopefully it isn't too hard to work with
- Mitigation: Deploy a "Hello World" site in Week 1 to identify pipeline issues.

Risk: Phaser Lifecycle

- Likelihood: Medium | Impact: Medium
- Evidence: Understanding when assets are loaded vs. created is new to the team.
- Mitigation: Review Phaser 3 "Scenes" documentation; prototype a simple sprite movement first.

Risk: Input Lag

- Likelihood: Medium | Impact: High
- Evidence: Web-based games can have varied input latency.
- Mitigation: Use Phaser's built-in update loop with delta time for consistency.

Risk: Collision Bugs

- Likelihood: High | Impact: High
- Evidence: Character getting stuck in text walls.
- Mitigation: Use Phaser's setCollisionByProperty on tilemaps to ensure rigid boundaries.

Risk 5: File Validation Logic

- Likelihood: Medium | Impact: High *
- Evidence: Users may upload .txt files with missing player spawns or nonsensical characters.
- Reduction Step: Implementing a strict character-to-tile legend in the Parser.
- Detection Plan: Automated unit testing of the Parser.js with corrupted text files.
- Mitigation: If validation fails, the UI will intercept the error and prompt the user to check their file against the "User Guide" instead of letting the game crash.

ii. Project Schedule

- Part 1: Setup Heroku and Node Boilerplate (1 Person-Week)
- Part 2: Phaser Scene and Asset Loading (1 Person-Week) - Dependent on part 1
- Part 3: String-to-Tilemap Parser (1 Person-Week)
- Part 4: Player Movement and Physics (1 Person-Week) - Dependent on part 2
- Part 5: File Upload Integration (1 Person-Week) - Dependent on part 3 and 4

iii. Team Structure

- Frontend/Phaser Lead: Game loop, scenes, and animations.
- Backend/DevOps Lead: Heroku deployment and Node.js server logic.
- Logic Lead: Parser development and text-to-coordinate mapping.
- QA/UI Designer: CSS for web wrapper and GitHub Issues/Bug tracking.

iv. Test Plan and Bugs

- Unit Testing: Write unit tests to better identify where problems exist (maybe use Jest? Recommended by Google and AI)
- System Testing: End-to-end testing in Chrome and Firefox.
- Usability Testing: Observe a peer trying to play our game, create their own level, see where they encounter issues.
- Bug Tracking: GitHub Issues will be made to track bugs/incomplete features

v. Documentation Plan

- Admin/Dev Guide: CONTRIBUTING.md explaining local setup (npm install/start).
- User Guide: An Instructions modal inside the game menu.
- API Reference: Documentation of the Parser's character-to-tile legend.

Example Directory setupid:

```
Textplorer/
├── public/            // Everything in here is what the browser sees
│   ├── assets/         // Images, audio, tilemaps
│   ├── levels/         // .txt level files
│   ├── src/           // Phaser game logic (Player.js, Scene.js)
│   └── index.html      // The entry point for Phaser
├── server.js          //  Node/Express server code
├── package.json        // Project metadata and dependencies
└── Procfile           // Specific instruction for Heroku
```

Test Automation

Infrastructure/Justification: We are using Jest for our unit and integration testing. This was chosen for its support of javascript and its built-in code coverage reporting. It also had a very simple setup and includes a zero config method which integrates easily with our phaser architecture.

How to add a new test:

1. Create a new xxxxx.test.js file in the /test_cases directory.
2. Import the new function to be tested
3. Use Jest's test() and expect() syntax to define the testing specifications
4. Run npm test locally to verify if they've past or not

Continuous integration service: Github Actions

Our repository is linked via the .github/workflows/node.js.yml file. This tells Github to execute npm test automatically when uploading a new file to play. We chose GitHub Actions because it is natively integrated into our system, allowing us to see the test result directly on commits and pull requests without the help of external sources.

Name	Pros	Cons
GitHub Actions	Native integration, free, easy setup	Might have slower start times since we're using the free version of it
CircleCI	Builds fast, great caching logic	Requires external account and more complicated configuration
TravisCi	Great for open source programs	Limited free version, slow integration

Tests executed in a CI build

- Every CI build executes our full file of unit tests for parser logic written in parser.test.js along with the validations tests for error handling

Triggers

- A CI build is triggered on every Push to any branch
- A CI build is triggered whenever a Pull Request is opened or updated

Reflections:

Jacob Munly:

1. Clear requirements definition: I learned that clearly defining requirements early is vital because vague feature descriptions can lead to different interpretations between team members. Spending more time clarifying requirements upfront limits reworking.

2. Team Communication: Regular and frequent communication between team members allowed for us to effectivly develop our system. When communication slowed down, progress also slowed because people were unsure about the status of certain features or what should be completed.

3. Testing gameplay mechanics under real scenarios: Some issues only appeared when the game was actually played rather than during development. This reinforced the importance of testing features in realistic scenarios to ensure they behave correctly during gameplay.

Jack Dunlap:

1. Iterative improvement through team feedback: Frequent and often feedback from teammates and testers helped refine features and improve the overall game. This allowed for updating and improving features to work much more smoothly, as opposed to features where feedback was limited they were harder to improve.

2. Code consistence and documentation: Working on a shared project showed how important a consistent coding style and documentation is. Establishing coding conventions early helped us have and understandable codebase, which was easier to maintain.

3. Managing branches and PR workflows: I learned how important it is to use a structured branching strategy when multiple developers are contributing to the same repository. Creating feature branches and merging changes through pull requests helped isolate new features, allowed for code review, and reduced the risk of introducing bugs into the main branch.

Drew Inglesby:

1. Time management and feature complexity: During development I noticed that some features seem simple at first but can require much more technical effort than expected. This made it clear how important it is to properly estimate the amount of time it may take to implement a feature and overestimate if needed.

2. Coordination between team members: I learned that understanding who is doing what within the team is vital to having a successful product. When responsibilities were clearly assigned, development moved more efficiently and teammates could focus on completing their tasks.

3. Learning professional GitHub workflows: Working on this project helped me better understand how GitHub is used in real collaborative software development. I learned how to manage feature branches, submit pull requests, review code changes, and track project progress through commits and issues.

Norman Nomie:

1. Team collaboration through GitHub: I learned a lot about working in a GitHub repository as a team. I have not had prior experience of working with a team in GitHub where we were all working on the same files. Prior to this class I did not know much about making new branches and submitting merge requests and I learned how to do that during this course.

2. Planning and breaking features down: Another very important thing I learned this term was planning and breaking down features of a large project into smaller tasks. When tasks were clearly scoped and described it was easy to coordinate with team members and improve the overall system efficiently.

3. Iterative development and testing: Building features like the Ghost Runner showed me how developing and testing functionality incrementally rather than all at once is very important. Testing small parts of the feature helped to catch bugs early and add fixes sooner.